

软件项目开发与实践

(pygame RPG 游戏) ——作业 2

课程名称： 软件项目开发与实践

学生姓名： 谢宇鹏

学 号： 5120243390

专业班级： 软件工程 2401

提交日期： 2026 年 6 月 28 日

项目名称： 星露谷动作 X (Stardew Valley Action X)

1 项目代码仓库与版本控制

1.1 公开 Git 仓库链接

本项目采用 Git 进行版本管理，托管于 GitHub 公开仓库，仓库内保留从 V1 到 V11 的全部历史版本：

<https://github.com/xieyupeng123/shixun>

1.2 提交（Commit）历史记录

以下为项目的完整提交历史截图，展示了从文档准备（6 月 23 日）到最终 V11 版本（6 月 28 日）的开发迭代全过程，总计 18 次提交，跨越 6 个自然日。

xieyupeng123 feat: Stardew Valley Action X V11 — 无敌模式重写：30秒自动过期 + 5分钟... 652454a · 11 hours ago 14 Commits		
fig	修复图表问题：PlantUML图预渲染为PNG后用includegraphic...	5 days ago
stardew-valley-action-x-final	feat: Stardew Valley Action X V11 — 无敌模式重写：30秒自...	11 hours ago
stardew-valley-action-x-v10	feat: Stardew Valley Action X V10 — 无敌模式 + 音效系统 + ...	2 days ago
stardew-valley-action-x-v3	feat: Stardew Valley Action X V3 — 完整架构 + 敌人AI	3 days ago
stardew-valley-action-x-v4	feat: Stardew Valley Action X V4 — 终版优化	3 days ago
stardew-valley-action-x-v5	feat: Stardew Valley Action X V5 — 真实怪物精灵 + 完整粒子...	3 days ago
stardew-valley-action-x-v8	feat: Stardew Valley Action X V8 终版 — Boss战 + 暂停存档 + ...	3 days ago
stardew-valley-action-x-v9	feat: Stardew Valley Action X V9 — 架构重构 + Boss系统 + 世...	3 days ago
stardew-valley-action-x	feat: Stardew Valley Action X V2 - 加入开始页面与基础攻击	3 days ago
讨论记录1-需求分析及设计方案.pdf	更新4篇文档：讨论记录改为通俗风格，需求文档和设计文档...	5 days ago
讨论记录1-需求分析及设计方案.tex	更新4篇文档：讨论记录改为通俗风格，需求文档和设计文档...	5 days ago
讨论记录2-AI对软件开发的影响及工具分享.pdf	更新4篇文档：讨论记录改为通俗风格，需求文档和设计文档...	5 days ago
讨论记录2-AI对软件开发的影响及工具分享.tex	更新4篇文档：讨论记录改为通俗风格，需求文档和设计文档...	5 days ago
设计与实现文档.pdf	修复图表问题：PlantUML图预渲染为PNG后用includegraphic...	5 days ago
设计与实现文档.tex	修复图表问题：PlantUML图预渲染为PNG后用includegraphic...	5 days ago
需求文档.pdf	修复图表问题：PlantUML图预渲染为PNG后用includegraphic...	5 days ago

图 1: Git 提交历史总览

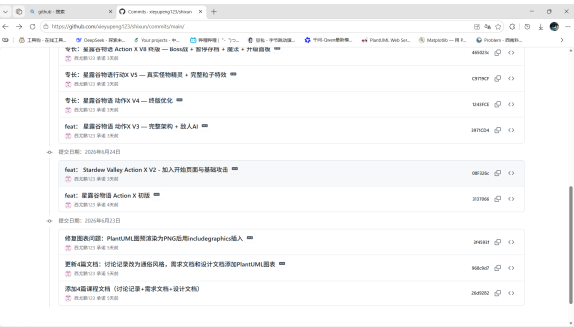


图 2: 提交详情一：文档阶段与初版至 V4

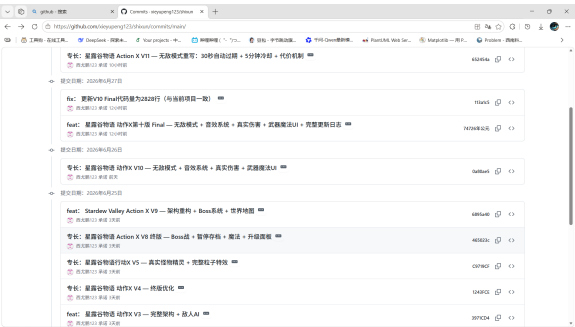


图 3: 提交详情二：V5 至 V10 版本

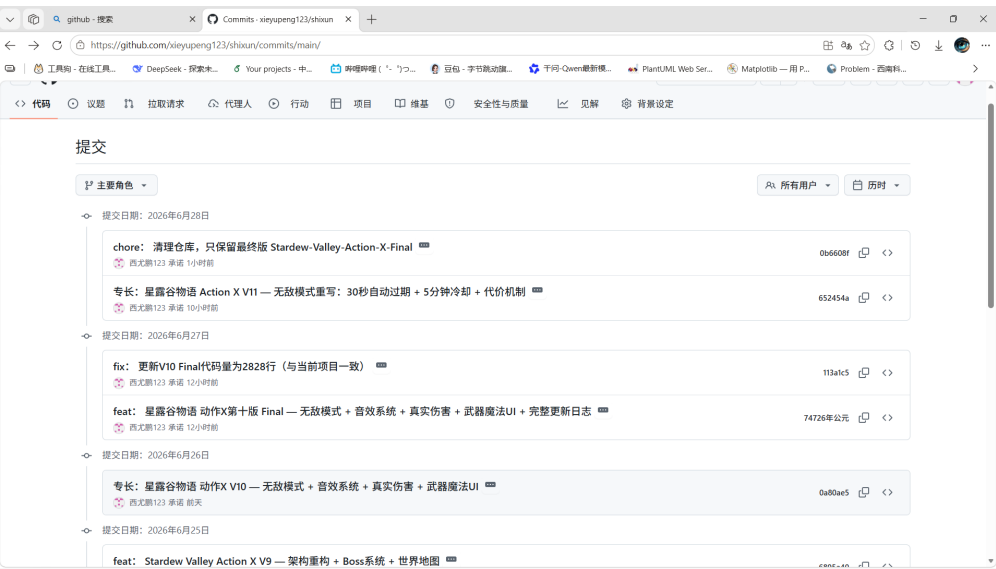


图 4: 提交详情三：V10 Final 至 V11 完整版

日期	Hash	版本	提交说明
06/23 14:20	26d9282	—	添加 4 篇课程文档（讨论记录 + 需求文档 + 设计文档）
06/23 16:00	968c9d7	—	更新 4 篇文档为通俗风格，添加 PlantUML 图表
06/23 16:16	3f4593f	—	修复 PlantUML 图预渲染为 PNG 后插入
06/24 17:03	3137066	V1	初版：基础窗口和角色显示
06/24 23:46	08f326c	V2	加入主菜单页面与基础攻击系统
06/25 08:19	3971cd4	V3	完整四层分层架构 + 敌人 AI 状态机
06/25 09:26	1243fce	V4	终版优化：代码重构与性能提升
06/25 10:43	c9719cf	V5	真实怪物精灵 + 完整粒子特效系统
06/25 18:54	465023c	V8	Boss 战 + 暂停存档 + 魔法释放 + 升级面板
06/25 23:12	6895a40	V9	架构重构：组合模式 + Boss 三阶段 + 世界地图
06/26 08:17	0a80ae5	V10	无敌模式 + 音效系统 + 真实伤害 + 武器魔法 UI
06/27 22:27	74726ad	V10F	V10 Final：完整更新日志
06/27 22:34	113a1c5	—	更新 V10 Final 代码量为 2828 行
06/28 00:33	652454a	V11	无敌模式重写：30 秒自动过期 +5 分钟冷却 + 代价机制
06/28 11:43	1754d54	—	添加提示词文档集合（系统提示词 + 各阶段开发指南）

表 1：项目完整提交历史详表

1.3 版本控制反思

在本次项目中，我采用的版本控制策略可以概括为「架构优先，增量迭代，大版本突变提交，本地同步备份」：

- (1) 架构优先，增量迭代。以 V3 为里程碑，首先建立四层分层架构的骨架。后续版本在此骨架上增量添加功能：V4 优化、V5 添加美术资源、V8 引入 Boss 和升级系统、V9 架构重构。每次改动都确保在已有架构框架内进行。
- (2) 工作粒度与功能完整度匹配。每个提交对应一个明确的功能里程碑：V8 一次性引入 Boss 战、暂停存档、魔法、升级面板四个系统，粒度虽大但功能自成闭环；V4 是纯代码优化，可以独立回溯。这种策略使每次提交都具有独立的可运行性和可测试性。

- (3) **本地同步备份，防范云端风险。**每次云端提交前确保本地通过测试验证。本地保有多份历史备份，为云端可能出现的误操作提供回旋余地。实践中确实遇到了远端仓库被清理的情况（提交 0b6608f），得益于充分的本地备份，所有历史版本得以完整恢复，未造成数据损失。
- (4) **持续集成的开发节奏。**提交跨越 6 天，每天集中开发并提交，实现了符合学生项目特点的持续开发节奏。

2 AI 协作能力专项测评

2.1 与 AI 结对编程——调试/功能实现实录

2.1.1 (1) 问题描述

在游戏开发中，我所遭遇的最棘手的技术难点是**多重 BGM 的切换循环冲突和覆盖问题**。该问题本质上是一个全局状态一致性问题：系统同时维护普通 BGM、Boss BGM、无敌音效三套音效通道，而切换逻辑散布在 Player、Level、Game 等多个模块中，缺乏统一的互斥保护。

我将问题归纳为以下五个典型场景：

场景	触发条件	表现	根因
1	Boss 击杀后调用 <code>_restart_game()</code> ， <code>MusicState</code> 仍为 1	普通地图播放 Boss 战斗音乐	<code>_restart_game()</code> 遗漏了 <code>MusicState.reset()</code> 调用
2	普通 BGM 下激活无敌 → 进入 Boss 区域 (<code>MusicState</code> →1)→ 无敌到期	BGM 未切换到 Boss 音乐，恢复了普通 BGM	<code>_end_invincibility()</code> 的路径判断逻辑有缺陷
3	无敌状态下冲入 Boss 区域 → 无敌到期	所有音效完全丢失，游戏无声	多个 <code>pygame.mixer.music</code> 操作缺乏互斥保护
4	Boss BGM 触发条件错误：必须受到 Boss 攻击才切换	即使已靠近 Boss，只要未受击就保持普通 BGM	触发逻辑使用了攻击事件而非 A* 寻路的探测范围
5	BGM 已切为 Boss 音乐后，后台每帧仍在轮询检测距离	Boss 战期间持续无效运算，造成 CPU 浪费	缺少已“触发则跳过的”短路判断机制

表 2: BGM 多层切换问题的五个典型场景

问题的核心在于 BGM 功能涉及多重变化和时机切换，而切换逻辑各自为政，没有统一的状态管理器。这类似于操作系统中**多个进程同时操作共享变量而没有引入互斥锁**导致的竞态条件问题。此外，BGM 触发条件和轮询效率也存在设计缺陷，整体形成了一个多层次、彼此交织的复杂问题网络。

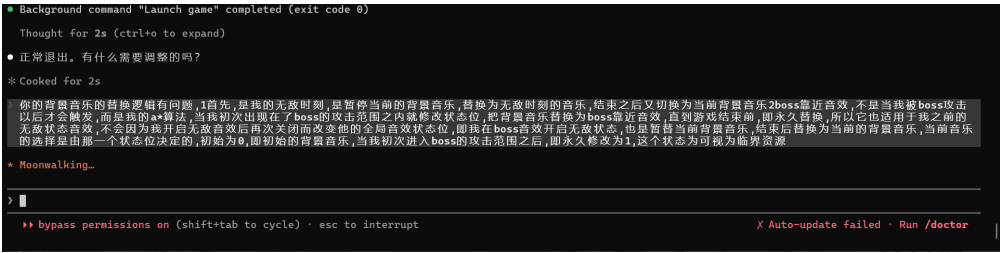


图 5: 向 AI 描述 BGM 设计方案和临界资源锁定问题

2.1.2 (2) 你的 AI 提示词

以下为调试过程中输入给 AI 的关键 Prompt 内容：

「我现在遇到了 BGM 多层切换引起的音效混乱问题。项目背景：普通地图播放 game_bgm.ogg；进入 Boss 区域后自动切换到 boss_bgm.ogg；按空格激活无敌模式时停止当前 BGM 并循环播放 invincible.ogg。

我发现了五个问题：(1) Boss 被击杀后重启游戏，普通地图仍在播放 Boss 音乐，请检查 _restart_game() 中 MusicState.reset() 是否被调用；(2) 无敌 → 进入 Boss 区域 → 无敌到期，BGM 切回了普通音乐而非 Boss 音乐，_end_invincibility() 的路径判断有缺陷；(3) 无敌状态挑战 Boss→ 无敌到期后所有音效丢失，怀疑 pygame.mixer 操作之间缺少互斥保护；(4) Boss BGM 触发条件当前是受到攻击才切换，应该改为进入 A*notice_radius 就触发；(5) BGM 已经切为 Boss 音乐后，后台还在每帧轮询距离检测，造成 CPU 浪费，需要加入短路判断。

请参考操作系统的信号量机制和互斥思想来重新设计 BGM 管理方案。要求：每次重启先重置状态量；BGM 加载必须通过 MusicState 的 get()/set()/reset() 三个原语中转；无敌模式只能激活不能手动关闭；首次检测到进入范围即永久锁定状态直到游戏重启。请给出具体代码修改方案。」

同时，我提供了带有时间戳的运行日志截图，帮助 AI 理解具体的状态转换和错误位置：

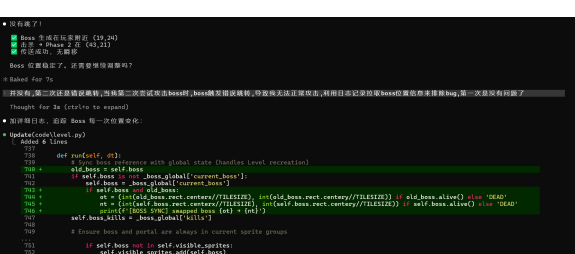


图 6: 通过日志拉取调试 BGM 状态问题

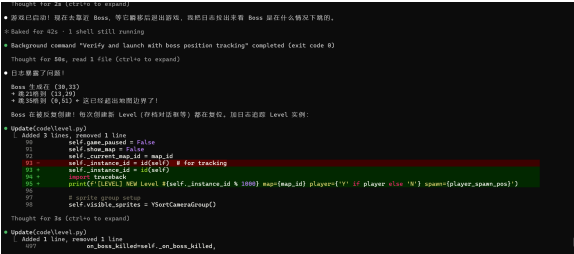


图 7: 日志分析后成功定位并修复 BGM 问题

2.1.3 (3) AI 给出的解决方案

AI 接收完整上下文后给出的初始方案如下（关键代码）：

```
1 def _end_invincibility(self):
2     self.invincible = False
3     if self.invincible_sound:
4         self.invincible_sound.stop()
5     # ---- AI's problematic code ----
6     if MusicState.get() == 1:
7         # Check if channel is free before loading
8         if not pygame.mixer.music.get_busy():
9             pygame.mixer.music.load(get_path('../audio/boss_bgm.ogg'))
10            pygame.mixer.music.set_volume(0.5)
11            pygame.mixer.music.play(-1)
12        else:
13            pygame.mixer.music.load(get_path('../audio/game_bgm.ogg'))
14            pygame.mixer.music.set_volume(0.3)
15            pygame.mixer.music.play(-1)
```

Listing 1: AI 初始方案的 BGM 恢复逻辑

AI 方案的致命缺陷：在 `MusicState.get()==1` 的分支中增加了 `get_busy()` 前置检查。当无敌音效正在循环播放时，`get_busy()` 返回 `true`，代码跳过 Boss BGM 的加载——结果 `MusicState` 已是 1 但实际音频为空，系统进入无声死锁。AI 没有意识到在多状态切换场景中，检查当前播放状态不能替代加载正确目标音频。

2.1.4 (4) 你的最终处理方式

C. AI 的方案有缺陷，我指出了问题 并让其修正，最终人机共同完成。

选择理由与修正过程：

我选择了选项 C。AI 的方案在大部分正常路径下是正确的，但在边界路径（无敌状态下进入 Boss 区域）存在由 `get_busy()` 引发的死锁缺陷。经过分析，我提出了以下修正策略——五个修正点分别对应上述五个问题场景：

- (1) 每次 `_restart_game()` 先无条件调用 `MusicState.reset()`，清空所有残留状态后再启动新游戏。同时调用 `ResourceManager.instance().stop_all_sounds()` 和 `pygame.mixer.music.stop()` 清空音频通道（对应场景 1）。
- (2) 去掉 `_end_invincibility()` 中的 `get_busy()` 死锁检查，改为「先无条件停止，再按 `MusicState` 信号量决定加载目标 BGM」，不依赖任何运行时状态判断（对应场景 2 和 3）。

- (3) 确立 `MusicState` 的三个原语为 BGM 唯一入口，禁止任何模块直接操作 `pygame.mixer.music`。这类似于操作系统中信号量原语的设计——所有对共享资源的访问必须通过统一的 P/V 操作（对应场景 2 和 3）。
- (4) 修改 Boss BGM 触发条件——从「受到 Boss 攻击」改为「首次进入 Boss 的 `notice_radius` 探测范围」。结合 A* 寻路算法中怪物的 `notice_radius`（Boss 为 400 像素），在 `_check_boss_music()` 中计算玩家与 Boss 的欧几里得距离，一旦距离小于探测半径即触发切换。Boss BGM 设为与普通 BGM 同等的全局优先级，因此可以被无敌模式正常暂停和恢复（对应场景 4）。
- (5) 加入短路判断避免无效轮询——在 `_check_boss_music()` 方法顶部增加 `if MusicState.get() == 1: return`。首次检测并切换后，后续所有帧的调用立即返回，不再执行距离计算。状态锁保持到 `_restart_game()` 调用 `MusicState.reset()` 才被清除，实现「一次性检测，永久锁定」（对应场景 5）。

以下是两个核心修正的最终代码：

```

1 def _check_boss_music(self):
2     """One-way switch: when player first enters boss notice
3     radius (A* range), permanently switch to boss music."""
4     # Short-circuit: already triggered, skip ALL checks
5     if MusicState.get() == 1:
6         return
7     boss = self.boss.boss
8     if boss and boss.alive():
9         boss_vec = pygame.math.Vector2(boss.rect.center)
10        player_vec = pygame.math.Vector2(self.player.rect.center)
11        dist = (player_vec - boss_vec).magnitude()
12        if dist <= boss.notice_radius:
13            MusicState.set(1)
14            if not self.player.invincible:
15                pygame.mixer.music.stop()
16                pygame.mixer.music.load(self._boss_bgm_path)
17                pygame.mixer.music.set_volume(0.5)
18                pygame.mixer.music.play(-1)

```

Listing 2: Boss BGM 检测与短路优化

```

1 def _end_invincibility(self):
2     self.invincible = False
3     self.invincible_start_time = None
4     # Set 5-min cooldown
5     self.invincible_cooldown_until = pygame.time.get_ticks() +
6     ↪ INVINCIBLE_COOLDOWN
7     # Cost: -50% current HP (min 1) + all energy drained
8     self.health = max(1, int(self.health * (1 - INVINCIBLE_COST_HEALTH)))

```



```
8     self.energy = 0
9     # ---- FIXED: unconditionally stop, then load by semaphore ----
10    if self.invincible_sound:
11        self.invincible_sound.stop()
12    if MusicState.get() == 1:
13        bgm_path = get_path('../audio/boss_bgm.ogg'); vol = 0.5
14    else:
15        bgm_path = get_path('../audio/game_bgm.ogg'); vol = 0.3
16    pygame.mixer.music.load(bgm_path)
17    pygame.mixer.music.set_volume(vol)
18    pygame.mixer.music.play(-1)
```

Listing 3: 修正后的 BGM 恢复逻辑

在人机协作中学到的最重要一课：

AI 目前还很难彻底取代软件工程师。它的编码能力很强，但存在根本性的短板——AI 不会做概要设计，无法自主地从全局架构出发审视系统。对于需求分析、概要设计、代码规范、架构设计，这正是软件工程师立足于 AI 时代、让 AI 服务于我们的核心能力所在。

AI 时代开发的关键原则：磨刀不误砍柴工——前期的详细设计不会耽误进度，反而使最终方案与设计理念高度吻合。此外，在 AI 改动文件之前必须本人亲自确认每个修改的范围和影响，不能过于懒惰。

2.1.5 多模态协作与 API 成本优化

在本项目的 AI 协作中，我认为自己做得最好的方面是多模型协作策略。不同 AI 模型的 API 价格和能力各有侧重，我根据任务类型建立了分层使用策略：

任务类型	选用模型	选择理由与应用场景
代码架构设计、Bug 调试、逻辑推理	DeepSeek V4 Pro	推理能力世界一流，支持 100 万 token 上下文。用于 BGM 状态机修复、战斗系统设计、无敌模式逻辑等复杂任务
图像识别、地图构建、多模态分析	MiMo V2.5/Pro	具备图文识别能力，可分析 PNG 像素通道。用于水面蓝色通道检测、敌人精灵切图验证等视觉任务
简单读取、代码格式化、文档整理	基础模型/MiMo V2.5	Token 价格低廉，适合读取 CSV 地图文件、格式化注释、版本日志生成等轻量任务

表 3: 多模型分层协作策略

此外，我编写了模型切换脚本（switch-model.ps1），只需一行命令即可切换模型并保留完整历史对话上下文，避免了手动修改配置文件导致对话丢失的问题。

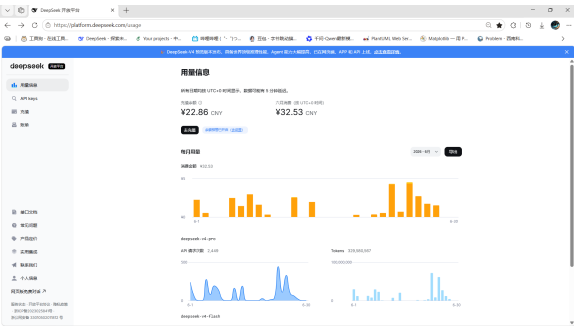


图 8: DeepSeek V4 Pro token 消耗统计

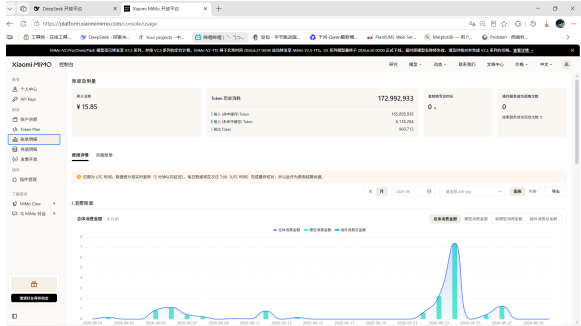


图 9: MiMo 系列 token 消耗统计

整个项目的 AI 协作总花费约 26 RMB（DeepSeek 约 15 RMB + MiMo 约 11 RMB），在控制成本的同时实现了高质量的多模型协作。

3 架构优化

在项目的实际开发中，我对系统架构进行了从 V1-V8（原始架构）到 V9-V11（优化架构）的大幅优化。核心优化方向是：通过引入**组合模式**和**单例模式**，遵循**单一职责原则**和**分层架构原则**，将 Level 从「上帝类」重构为「协调者」。

3.1 优化前架构

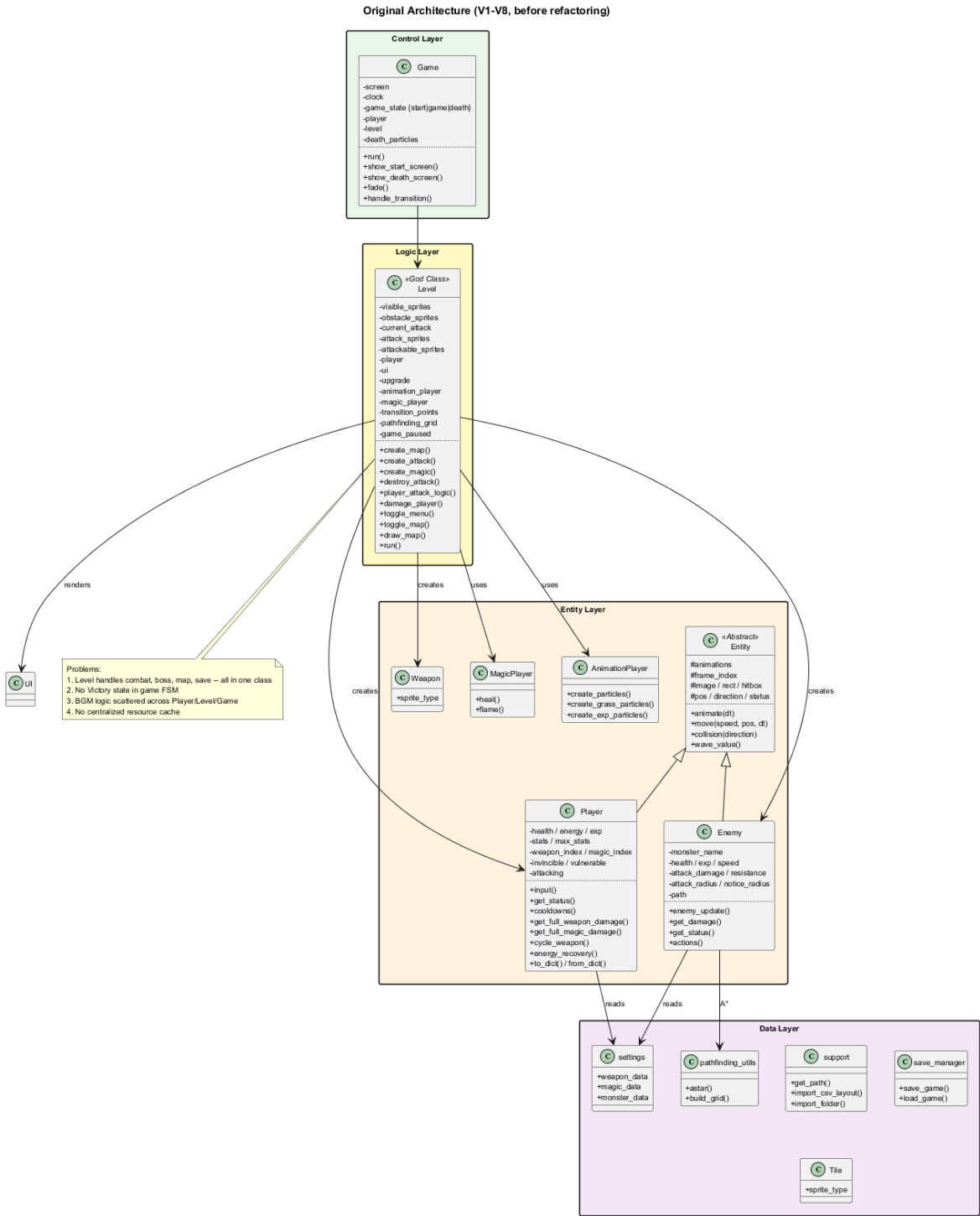


图 10: 系统优化前概要类图

原始架构的核心问题:

问题 1: Level 为 God Class (500+ 行) ——战斗逻辑、Boss 管理、地图渲染、实体创建、传送检测、存档序列化全部堆在一个类中, 违反 SRP 原则。任

何功能修改都需要触碰这个巨型类。

- 问题 2：缺少统一资源管理——图片、音效、字体各自独立加载，同一文件可能被多次加载造成内存浪费。加载失败没有统一降级机制。
- 问题 3：BGM 状态管理缺失——切换逻辑散布在 Player、Level、Game 三个模块中，各自直接操作 pygame.mixer，导致竞态条件和状态不一致。
- 问题 4：状态机不完整——仅有 start/game/death 三种状态，缺少 Victory 结算界面。

3.2 优化后架构

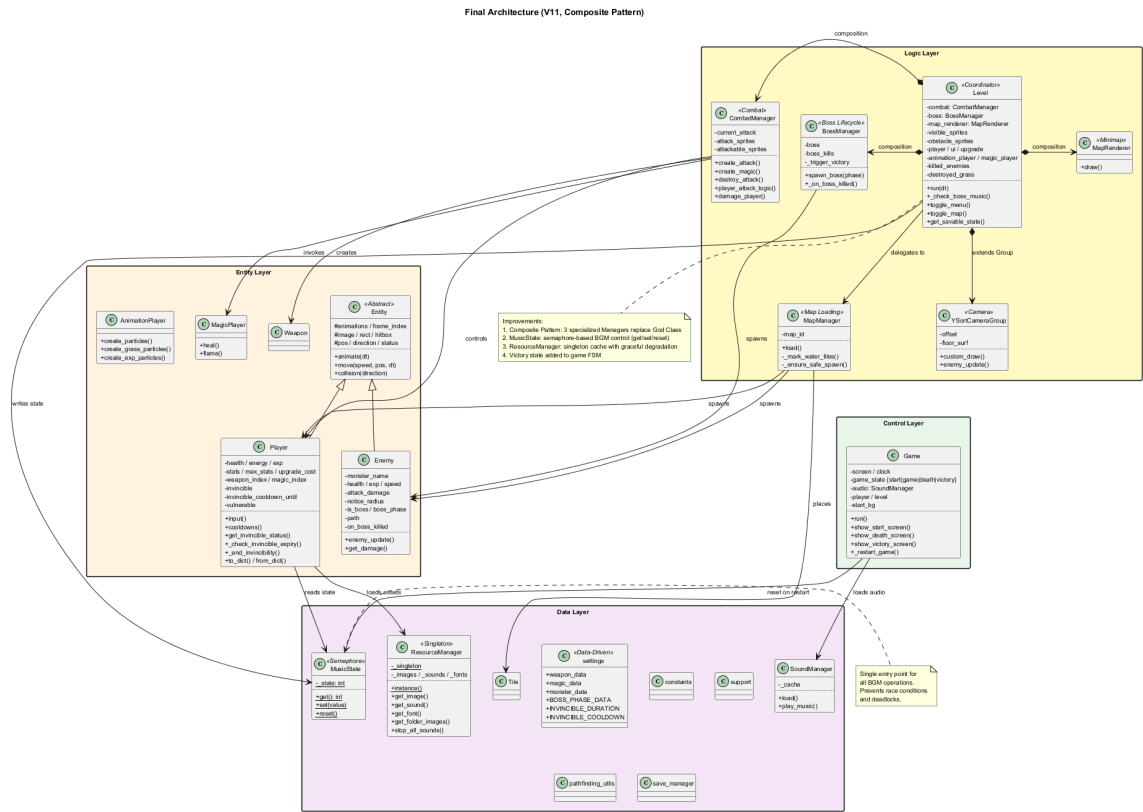


图 11：系统优化后概要类图

优化后的关键改进：

- 优化 1：引入组合模式——将 Level 拆分为 CombatManager（战斗调度，约 80 行）、BossManager（Boss 生命周期，约 60 行）、MapRenderer（地图叠加层，约 100 行）三个专职 Manager，Level 通过 @property 代理保持向下兼容。
- 优化 2：引入 MusicState 单例(信号量机制)——所有 BGM 操作统一经过 get()/set()/reset() 三个原语，禁止直接操作 pygame.mixer，从根本上解决多层切换的死锁和丢失问题。

- 优化 3：引入 **ResourceManager 单例模式**——全局唯一资源缓存实例，图片/音效/字体统一管理、按路径缓存、失败静默降级。
- 优化 4：新增 **Victory 状态**——游戏状态机完整覆盖 start→game→(death | victory) 的全生命周期。
- 优化 5：引入 **MapManager**——将 CSV 地图加载、四层瓦片铺设、水域像素检测、寻路网格构建全部封装为独立类，并实现水域检测结果缓存。

3.3 BGM 状态机设计

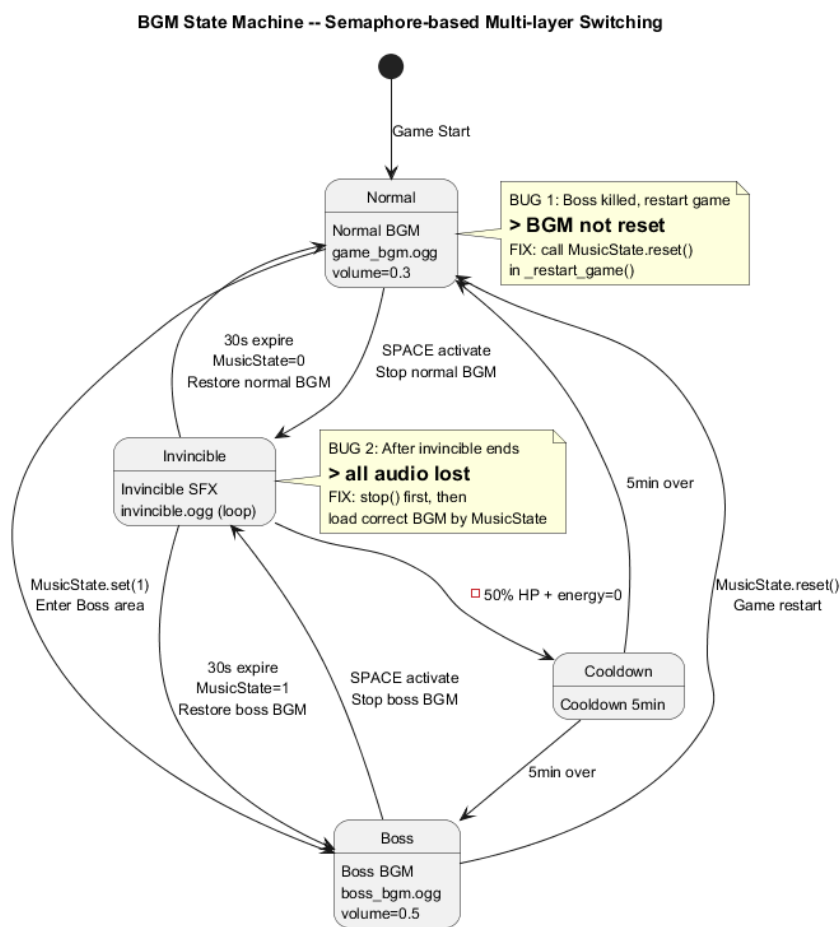


图 12: BGM 状态机

该状态机的设计灵感来源于操作系统中的**信号量**概念：MusicState 作为一个二值信号量 (0/1)，所有 BGM 消费者必须通过统一原语读写状态，且 reset() 操作在任何重启路径上始终被保证执行。通过这种方式，MusicState 的内部值与 pygame.mixer.music 的实际播放内容始终保持一致。

3.4 设计原则与设计模式

原则/模式	在本项目中的具体体现	带来的收益
单一职责原则	Level 拆分为 CombatManager、 BossManager、MapRenderer	每个类只有一条需要变更的理由，修改独立
组合模式	Level 通过组合引用集成各 Manager，而非继承	新增功能域只需新建 Manager 并组合，无需修改已有代码
单例模式	ResourceManager 和 MusicState 各只有一个全局实例	资源缓存统一、BGM 状态全局唯一
开闭原则	武器/怪物/魔法属性集中在 settings.py 字典中	新增类型只需加配置，不需改 核心逻辑
分层架构	控制层 → 逻辑层 → 实体层 → 数据层，单向依赖	依赖方向可控，便于独立测试 和替换底层
数据驱动设计	所有游戏数值集中于 settings.py 配置文件	策划调整数值无需开发者介入； 配置即文档

表 4: 架构优化采用的设计原则和设计模式

我的代码架构几乎在项目一开始（V3 阶段）就确定好了四层分层结构，后续的音乐环节只引入了几个额外类（MusicState、SoundManager），在保持核心架构不变的前提下优化了耦合度。这与软件工程中「架构先行、迭代优化」的理念一致——核心架构在前期设计时敲定，之后的每次迭代都在稳固的框架下进行增量优化。